FSF-Guided Program Slicing for Testing-Based Formal Verification of Functional Soundness and Completeness

Yang Liu[a], Zedong Jia[a], Xi Hu[b], Dhawaj Kumar Tomar[a], Ai Liu[a,*], Shaoying Liu[c]

[a] *College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing, Jiangsu, China*
[b] *School of Computing, The Australian National University, Canberra, Australian Capital Territory, Australia*
[c] *Software Engineering Institute, East China Normal University, Shanghai, China*

Abstract

Soundness and completeness are two sides of the same coin, used in program analysis to balance between catching all errors and avoiding false positives. Testing-based formal verification has been proposed to reduce testing costs and improve software reliability by automatically verifying the correctness of program execution paths with regard to the functional scenario form of a specification. In this paper, we propose the notions of functional soundness and functional completeness with regard to a functional scenario and develop the corresponding analysis techniques by integrating testing-based formal verification with program slicing. To further reduce the program scale and verification cost, we propose an FSF-guided program slicing technique, which generates a scenario-specific executable slice for each functional scenario (T_i, D_i). The technique identifies the input and output variables involved in the functional scenario, preserves their relevant data and control dependencies, and removes branches that are unreachable under the testing condition T_i.

Moreover, we implement tool support for the Java language and conduct an experiment on 250 Java programs containing 740 scenario-specific slicing tasks. The experiment investigates three research questions concerning the reduction in program scale, the reduction in verification cost, and the preservation of verification results. The experimental results show that FSF-guided slicing reduces the numbers of lines of code, executable statements, and cyclomatic complexity by 39.21%, 46.81%, and 45.81% on average, respectively. It also reduces the total cost of testing-based formal verification by 22.5%, achieves a 97.03% conclusive preservation rate over all tasks, and preserves the functional soundness and functional completeness judgments on all definite verification tasks.

Keywords: Functional scenario, Functional soundness, Functional completeness, Testing-based formal verification, Program slicing

1. Introduction

The essence of software engineering is to transform ideas (which are often vague and subjective) into programs. However, since a program is a formal entity with a precise functional implementation, the transition from ideas to programs should involve a formalisation at some point. The earlier this point is, the more benefits there are, such as formal concepts on an abstract level, unambiguous communication of ideas, and improving user trust. Formal methods for software engineering are a way to deal with this in a rigorous process Roggenbach et al. (2022).

Formal methods provide a structured approach to software development, with formal specification describing the software's expected behavior and properties, and verification ensuring that these specifications

*Corresponding author

Email addresses: lywww0310@nuaa.edu.cn (Yang Liu), nalanzed@gmail.com (Zedong Jia), xi.hu@anu.edu.au (Xi Hu), dhawajtomar@outlook.com (Dhawaj Kumar Tomar), shaoai@nuaa.edu.cn (Ai Liu), syliu@sei.ecnu.edu.cn (Shaoying Liu)

are satisfied, thus guaranteeing the software's reliability and safety Peled (2001). The verification techniques mainly include model checking Baier and Katoen (2008) and theorem proving Schumann (2001). Model checking can be computationally expensive and limited to smaller models, while theorem proving can be complex, time-consuming, and may not always provide a conclusive answer due to the undecidability of some logical systems.

Testing-based formal verification (TBFV) combines specification-based testing and formal verification to reduce test costs as well as increase automation of the verification process. The initial idea of TBFV, proposed by Liu (2012), adopts Hoare logic Hoare and Wirth (1973) to ensure the correctness of program paths (unfolding loop structures) derived by test cases. The idea is further developed to combine with program inspection Liu and Nakajima (2013). A collection of prototype tools to illustrate how to support the method is presented in Liu (2016). The TBFV approach is applied to the verification of SysML activity diagrams Yin et al. (2018). The original framework of TBFV is extended in Liu and Liu (2023) to deal with operations, which may involve complex data structures and side effects. Replacing the application of Hoare logic with symbolic execution, testing-based formal verification with symbolic execution (TBFV-SE) is proposed in Wang and Liu (2018). Branch sequence coverage (BSC) for TBFV-SE is considered in Wang and Liu (2019) and a fault localization approach is proposed to further pinpoint the problematic positions in the incorrect program paths given by TBFV-SE in Wang et al. (2020). Moreover, the idea is also applied to improve the verification of neural networks, resulting in TBFV-NN, a framework for verifying and improving neural networks Liu et al. (2022, 2024).

Functional scenario form (FSF), a transformation of formal specification, is used for formal specification animation Liu (2015) and test case generation in TBFV Liu and Nakajima (2022). An FSF is a disjunction of functional scenarios, any of which consists of a testing condition and a defining condition. A testing condition is a constraint on some input variables and is used to generate test cases. A defining condition is a constraint that should involve at least one output variable to specify the expected function. In short, functional soundness ensures that the function is correctly implemented for all possible inputs (avoiding false positives) while functional completeness guarantees that all possible outputs can be generated (avoiding false negatives).

When we focus on a functional scenario whose testing condition involves only a subset of input variables, it is unnecessary to analyze the entire program. Instead, the analysis can be restricted to the statements that are affected by these scenario-relevant input variables and may further contribute to the output variables appearing in the defining condition. Program slicing is an effective technique to extract the part of the program that can affect the values computed at a given program point (a.k.a. slicing criterion) Galindo et al. (2023b); Weiser (1984). An integral solution merging together the current program slicing extensions used to slice programs with exception-handling is proposed in Galindo et al. (2023a). Program slicing has been applied in many disciplines. For instance, an integration of program slicing with cognitive complexity for defect prediction is proposed in Alqadi and Maletic (2020). An empirical analysis on locating faults with program slicing is depicted in Soremekun et al. (2021). A debugging approach for runtime exceptions based on program slicing and stack traces is proposed in Jiang et al. (2010). A fault localization approach for null pointer exceptions based on stack trace and program slicing is developed in Jiang et al. (2012). A novel program segment testing technique for detecting potential occurrences of runtime exceptions during the program construction process is described in Rao et al. (2022).

In this paper, we develop a methodology to analyze the program implementation corresponding to a functional scenario with the notions of functional soundness and functional completeness by integrating testing-based formal verification and program slicing. This article substantially extends our previous conference paper Liu et al. (2025). The conference version introduced the notions of functional soundness and functional completeness, integrated testing-based formal verification with output-oriented program slicing, and reported a preliminary experiment involving 20 functional scenarios. Compared with the conference version, this article proposes an FSF-guided program slicing technique for constructing scenario-specific executable slices, extends the corresponding methodology and tool implementation, and conducts a substantially larger empirical evaluation on 250 Java programs containing 740 functional scenario slicing tasks. Our main contributions are as follows:

- We propose the notions of functional soundness and functional completeness with regard to a functional scenario.
- We propose an FSF-guided program slicing technique to construct scenario-specific program slices.
- We develop a general methodology to verify functional soundness and completeness of a functional scenario by integrating program slicing and testing-based formal verification.
- We implement tool support for the Java language and conduct a large-scale empirical evaluation on 250 Java programs containing 740 functional scenario slicing tasks. The evaluation investigates three research questions concerning the reduction in program scale, the reduction in verification cost, and the preservation of verification results.

The remainder of this paper is organized as follows. Section 2 explains the motivation for this work. Section 3 introduces the preliminary knowledge of program slicing. Section 4 defines the notions of functional soundness and functional completeness. Section 5 presents the proposed methodology. Section 6 reports the tool implementation and empirical evaluation. Section 7 discusses the threats to the validity of the empirical results. Section 8 reviews the studies related to testing-based formal verification, functional scenario forms, and program slicing. Section 9 concludes this paper and discusses directions for future work.

2. Motivation

Incorrectness logic O'Hearn (2020), a formalism similar to Hoare logic, provides a principled logical system to reason about the presence of bugs and also needs to deal with loop structures manually. Although incorrectness logic was proposed in 2020, it has attracted the attention of many researchers (more than 57 citations) and derived many variants, such as incorrectness logic for graph programs Poskitt (2021) and quantum programs Feng and Li (2023); Yan et al. (2022).

Hoare logic is based on specifications of the over-approximated triple

$$\{\textit{pre-condition}\}\ \textit{program}\ \{\textit{post-condition}\}$$

which states that the *post-condition* over-approximates (describes a superset of) the set of states reachable upon program termination from states that satisfy the *pre-condition*. Conversely, incorrectness logic uses the under-approximated triple

$$[\textit{presumption}]\ \textit{program}\ [\textit{result}]$$

which states that the *result* under-approximates (describes a subset of) the set of final states that can be reached starting from states satisfying the *presumption*. Example 1 shows that Hoare logic with over-approximated triples may result in false positives: reports of bugs that cannot happen. Example 2 illustrates that incorrectness logic with under-approximated triples may lead to false negatives (missed bugs).

Example 1. Consider the over-approximated triple of a method

$$\{x \in \mathbb{Z}\}$$
```
int f(int x){ return x*x+1;}
```
$$\{f(x) \geq 0\}$$

where $\mathbb{Z}$ is the domain of `int` type and the statement invoking the method `int y=10000/f(x)`. Obviously, the triple meets Hoare logic. If we analyze the statement with it, it seems possible to trigger an arithmetic exception since $f(x)$ can be 0 in the post-condition. However, this exception will never happen.

Example 2. Consider the under-approximated triple of a method

$$[x \in \mathbb{Z}]$$
```
int g(int x){ return x*x;}
```
$$[g(x) > 0 \wedge g(x) \in \{n^2 \mid n \in \mathbb{Z}\}]$$

and the statement invoking the method `int y=10000/g(x)`. Obviously, the triple meets incorrectness logic. If we analyze the statement with it, it can be proved that the arithmetic exception will not occur since $g(x)$ cannot be 0 in the result. However, this exception may occur when the input x is 0. Note that if we remove $g(x) \in \{n^2 \mid n \in \mathbb{Z}\}$ from the result, the triple will be neither under-approximated nor over-approximated.

However, reasoning with loop structures in both Hoare logic and incorrectness logic is difficult to fully automate and requires experts in logic to handle them manually. Moreover, the final pre-condition or result will be complex due to the loop invariants. Testing-based formal verification is an alternative approach proposed to unfold loop structures when executing the program with concrete test cases to automatically verify over-approximated triples to some extent. Over-approximated and under-approximated triples correspond to the notions of soundness and completeness, respectively. In this paper, we consider the notions of soundness and completeness for a functional scenario with regard to a program and further explore the utilization of TBFV for automatic verification on them.

3. Preliminaries

Program slicing is a sophisticated program analysis technique aimed at extracting relevant parts of a program that pertain to a particular computation or point of interest, known as the slicing criterion. This method is widely utilized in debugging, understanding, testing, and optimizing code by isolating segments that directly influence specific variables or output. The complexity and vastness of modern software systems require such techniques to streamline the process of program comprehension and fault isolation. In this section, we will adapt program slicing to obtain an expected program slice in our methodology.

A critical component in program slicing is the program dependency graph (PDG). A PDG is a directed graph representing both data and control dependencies between the statements and predicates within a program. It serves as a comprehensive model to visualize how data flows and how control decisions are made in the program. The basis of the PDG is the control flow graph (CFG) of a program, which is a graph $\langle N, E\rangle$ ($E \subseteq N \times N$) that can derive all possible execution paths of a program. Each statement is represented by a node $n \in N$, and two nodes are connected, i.e., $(m, n) \in E$, if they may be executed sequentially. Additionally, two nodes, *Enter* and *Exit* , are added to represent the initial and final nodes of the program execution, respectively. There are several definitions based on the CFG.

Definition 1. Given a CFG $\langle N, E\rangle$ and a node $n \in N$, the set of all the program variables that are defined (assigned with values) in n is denoted by $DEF(n)$.

Definition 2. Given a CFG $\langle N, E\rangle$ and a node $n \in N$, the set of all the program variables that are used (accessed with values) at n is denoted by $USE(n)$.

Definition 3. Given a CFG $\langle N, E\rangle$ and two nodes $m, n \in N$, we say that n post-dominates m if every directed path from m to the *Exit* node passes through n. We say that n is control dependent on m if and only if n post-dominates one but not all of m's successors, denoted by $m \to n$.

Definition 4. Given a CFG $\langle N, E\rangle$ and two nodes $m, n \in N$, we say that n is data dependent on m if there exists a variable v satisfying:

- $v \in DEF(m)$;
- $v \in USE(n)$;
- there exists a path in $\langle N, E\rangle$ from m to n where v is not redefined. In particular, this case is denoted by $m \xrightarrow{v} n$.

With the above definitions, we can define the notion of PDG and calculate a program slice for a slicing criterion.

Definition 5. Given a CFG $\langle N, E\rangle$ and a program P with $Var(P)$ denoting the set of variables in P, the corresponding PDG is a graph $\langle N', E', \phi\rangle$ where

- $N' = N - \{Exit\}$;
- $(m, n) \in E'$ if n is data dependent on m (the edge is called a data arc) or n is control dependent on m (the edge is called a control arc).
- $\phi : E' \to \mathcal{P}(Var(P))$ is a function such that

$$\phi((m,n)) = \begin{cases} \emptyset & n \text{ is control dependent on } m \\ \{v \mid m \xrightarrow{v} n\} & \text{otherwise} \end{cases}$$

Definition 6. Given a PDG for a program P and slicing criterion (k, V) where $V \subseteq DEF(k) \cup USE(k)$, the program slice denoted by $PS(P, (k, V))$ is defined as

$$PS(P, (k, V)) = \{m | (m \to^* \xrightarrow{v} k \wedge v \in V) \vee (m \to^* \to k)\}$$

where $m \to^*$ represents the set of all descendant nodes of m (including itself).

4. Functional Soundness and Completeness

Our methodology is based on the notion of functional scenario. Given a functional specification, we should first transform it into functional scenarios in our methodology. However, this step is not considered in this document, and there already exists an automatic transformation from VDM operation specifications to functional scenarios in Liu et al. (2010). In this section, we first introduce the notions of functional scenario, functional soundness, and functional completeness. Note that the notion of functional scenario in Definition 7 is different from the one in Liu and Liu (2023); Liu (2012), since our methodology considers both functional soundness and completeness, as in Definition 8.

Definition 7 (functional scenario)**.** A functional scenario is a pair of a testing condition T, a constraint restricted only on input variables in the specification and a defining condition D, restricted on at least one output variable, denoted by (T, D).

A testing condition T represents part of the inputs, and the corresponding D describes the expected behavior by restricting some output variables. The functional scenario only describes the functional behavior, but ignores to what extent it has been implemented, which is captured by the notions of functional soundness and completeness.

Definition 8 (functional soundness and functional completeness)**.** Given a functional scenario (T, D), where T is restricted on the input variables $x_1, \cdots, x_n$ and the output variables in D are $y_1, \cdots, y_m$, let $\mathtt{x} = (x_1, \cdots, x_n)$ and $\mathtt{y} = (y_1, \cdots, y_m)$. Given a program P, let $P(\mathtt{x})$ be the result of the output variables by executing P with $\mathtt{x}$ and $D(P(\mathtt{x})/\mathtt{y})$ the result by replacing $\mathtt{y}$ with $P(\mathtt{x})$ in D. Then,

- (T, D) is called sound with regard to P if the implication

$$T \Rightarrow D(P(\mathtt{x})/\mathtt{y})$$

  is a tautology;
- (T, D) is called complete with regard to P if the implication

$$\exists \mathtt{x}\ (T \wedge D) \Rightarrow \exists \mathtt{x}\ (T \wedge \mathtt{y} = P(\mathtt{x}))$$

  is a tautology.

One may be confused about why the premise of completeness is $\exists \boldsymbol{x}\ T \wedge D$ rather than D. Since the definition of D does not have a restriction on input variables, D may contain some input variables. Using $\exists \boldsymbol{x}\ T \wedge D$, the free variables of the implication will only be the output variables.

The notions of functional soundness and completeness correspond to the over-approximated and under-approximated triples, respectively, as described in Theorem 1.

Theorem 1. *Given a functional scenario (T, D) and a program P, the following hold:*

- *the functional scenario (T, D) is sound with regard to P if and only if $\{T\}P\{D\}$ is an over-approximated triple;*
- *the functional scenario (T, D) is complete with regard to P if and only if $[T]P[D]$ is an under-approximated triple.*

As we mentioned earlier, a specification should be transformed into a functional scenario family in our methodology. Note that two functional scenarios (T, D_1) and (T, D_2) are equivalent to $(T, D_1 \vee D_2)$ since they describe the same functional behavior; i.e., if the input validates the testing condition T, the output is expected to validate either D_1 or D_2. Obviously, the functional soundness (completeness) of (T, D_i) $(i = 1, 2)$ may be different from $(T, D_1 \vee D_2)$. Therefore, we should explore the relation among functional scenarios further, resulting in Definition 9.

Definition 9 (functional scenario family)**.** A functional scenario family is a set of functional scenarios, denoted by $\{(T_1, D_1), \cdots, (T_l, D_l)\}$. Specifically,

- it is called a functional scenario family with mutually exclusive input if $T_i \wedge T_j$ is a contradiction when $i \neq j$;
- it is called a functional scenario family with mutually exclusive output if $D_i \wedge D_j$ is a contradiction when D_i and D_j have the same variables and $i \neq j$.

Theorem 1 demonstrates the correspondence between functional soundness (completeness) and over-(under-)approximated triples. An over-approximated triple strictly requires that the post-condition holds after the program is executed if the pre-condition is validated. If there are other possible outputs for the inputs validating the testing condition, the functional soundness of the functional scenario does not make sense, as demonstrated in Example 3. Therefore, when we consider the functional soundness of a functional scenario belonging to a functional scenario family, we should ensure the functional scenario is with mutually exclusive input at first.

Example 3. Continuing Example 2, it is obvious that

$$(x \in \mathbb{Z}, g(x) > 0 \wedge g(x) \in \{n^2 \mid n \in \mathbb{Z}\})$$

is unsound with regard to the program `int g(int x){ return x*x;}` and $(x \in \mathbb{Z}, g(x) = 0)$ is unsound with regard to the same program. However, if the specification derives the two functional scenarios, the functional behavior should describe that the program returns the square of the integer input. Therefore, separately verifying the functional soundness of the two functional scenarios does not make sense. The functional scenario to be considered should be

$$(x \in \mathbb{Z}, g(x) \geq 0 \wedge g(x) \in \{n^2 \mid n \in \mathbb{Z}\}),$$

which is sound with regard to the program.

As for functional completeness, considering a functional scenario family $\{(T_1, D), (T_2, D)\}$ induced by a certain specification, the corresponding functional behavior requires that the inputs validate T_1 or T_2 and the outputs validate D. Naturally, when it comes to completeness, we should find at least one input validating $T_1 \vee T_2$ for each output validating D, so the functional scenario to be verified should be $(T_1 \vee T_2, D)$. Therefore, when we consider the functional completeness of a functional scenario belonging to a functional scenario family, we should ensure the functional scenario is with mutually exclusive outputs in advance.

5. Methodology

As introduced in Section 4, the prerequisite of our methodology is a functional specification–functional scenario family $\{(T_1, D_1), \cdots, (T_l, D_l)\}$ with mutually exclusive input and output, where T_i is the constraint restricted only on input variables and D_i the constraint restricted on at least one output variable. Given a functional scenario (T, D), we use a vector $\boldsymbol{x} = (x_1, \cdots, x_n)$ to denote all input variables appearing in T or D, and a vector $\boldsymbol{y} = (y_1, \cdots, y_m)$ to denote all output variables appearing in D.

5.1. An Overview

Given a functional scenario, we intend to analyze its soundness and completeness by integrating testing-based formal verification with program slicing. An overview of our methodology is depicted in Fig. 1. There are three stages: slicing stage, testing stage, and verification stage.

- In the slicing stage, with the FSF-guided program slicing technique, we will obtain a scenario-specific program slice for each functional scenario (T, D). The testing condition T is used to restrict the feasible paths and identify scenario-relevant computations, while the defining condition D is used to determine the output behavior that should be preserved. The resulting slice is a new executable program. This stage aims to reduce the program scale and thus reduce the cost of testing-based formal verification.

- In the testing stage, based on the obtained program slice, we will generate test cases $t_1, \cdots, t_k$, resulting in different program execution paths $path_1, \cdots, path_k$. For each $path_i$, we will derive the corresponding path condition C_i, which is a constraint restricted only on input variables and represents the inputs resulting in the same path $path_i$, and the corresponding state representation of the output variables $f_i : C_i \wedge T \to Y_1 \times \cdots \times Y_m$ (Y_i is the domain of y_i), which maps an input $\boldsymbol{x}$ to the output of executing the program slice with it, denoted by $\boldsymbol{y} = f_i(\boldsymbol{x})$.

- In the verification stage, with the above ingredients, we can verify functional soundness and completeness, respectively. For functional soundness, the subsequent steps are as follows.

  - Let $D(f_i(\boldsymbol{x})/\boldsymbol{y})$ be the result by replacing the output $\boldsymbol{y}$ with $f_i(\boldsymbol{x})$ in D and thus its free variables will only include $x_1, \cdots, x_n$.
  - If the implication $\bigwedge_{i=1}^{k}(T \wedge C_i \Rightarrow D(f_i(\boldsymbol{x})/\boldsymbol{y}))$ is not a tautology, it means that there exists at least one input such that the output is not as expected, i.e., the program implementation is unsound with regard to the functional scenario (T, D).
  - If the above implication is a tautology, the other implication $T \Rightarrow C_1 \vee \cdots \vee C_k$ is to be considered: if it is a tautology, the program implementation is sound with regard to the functional scenario (T, D); otherwise, the program implementation is locally sound for inputs validating $(C_1 \vee \cdots \vee C_k) \wedge T$.

  For functional completeness, the subsequent steps are as follows.

  - The free variables of the implication $\exists \boldsymbol{x}\ (T \wedge D) \Rightarrow \exists i \exists \boldsymbol{x}\ (T \wedge C_i \wedge \boldsymbol{y} = f_i(\boldsymbol{x}))$ will only include $y_1, \cdots, y_m$. If it is a tautology, the program implementation is complete with regard to the functional scenario (T, D).
  - If the above implication is not a tautology, we further check whether $T \Rightarrow C_1 \vee \cdots \vee C_k$ is a tautology. If it is, the explored path conditions cover all inputs satisfying T, and thus the program implementation is incomplete with regard to (T, D). Otherwise, the generated path conditions cover only part of T, so no global completeness judgment can be made; the result is local to the explored input region $T \wedge (C_1 \vee \cdots \vee C_k)$.

5.2. FSF-Guided Program Slicing

As illustrated in Fig. 1, the desired program slice in our methodology is a scenario-specific slice corresponding to each functional scenario (T_i, D_i). In FSF-guided slicing, the testing condition T_i is used to identify scenario-relevant input variables and feasible branches, while the defining condition D_i specifies the output behavior that should be preserved. Therefore, we first identify the input locations of the variables appearing in T_i and the export locations of the output variables appearing in D_i. Then, based on the PDG, we retain the statements that are relevant to the dependence paths from these input variables to the corresponding output variables under the constraint of T_i. Finally, these statements are integrated into an executable program. It is concluded in Definition 10.

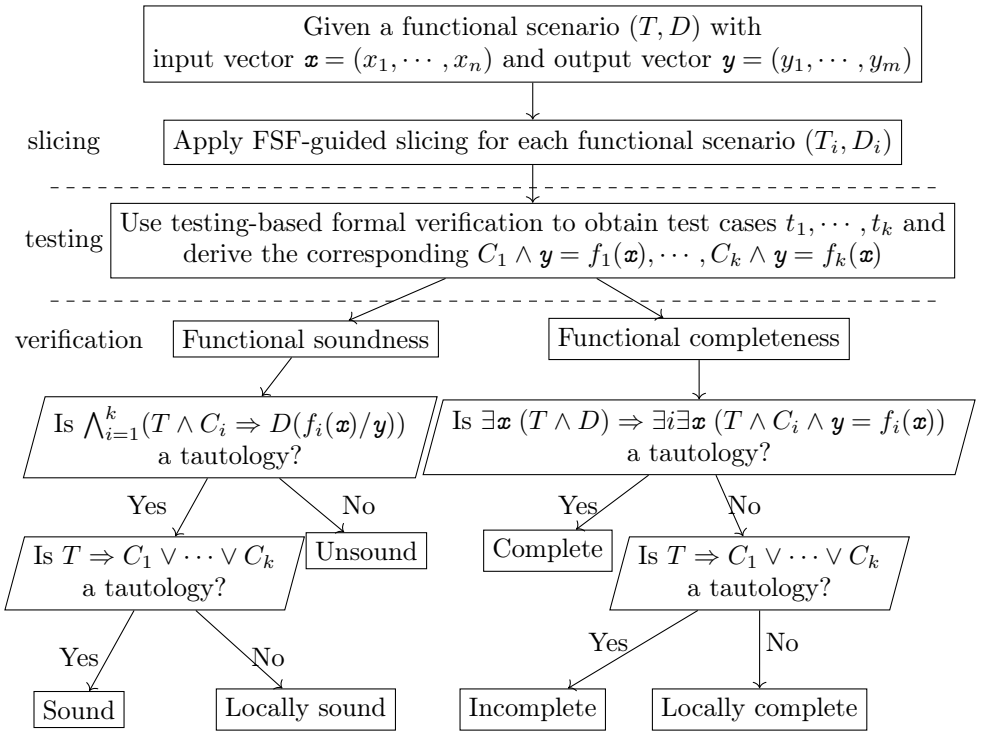

Figure 1: The overview of our methodology

Definition 10. Given a PDG for a program P and a functional scenario (T_i, D_i), let X_{T_i} be the set of input variables appearing in T_i, let X_{D_i} be the set of input variables appearing in D_i, and let Y_{D_i} be the set of output variables appearing in D_i. We denote the set of scenario-relevant input variables by $X_i = X_{T_i} \cup X_{D_i}$. Denote the input locations of each $x \in X_i$ as $p_x^1, \cdots, p_x^{r_x}$, and the export locations of each $y \in Y_{D_i}$ as $l_y^1, \cdots, l_y^{k_y}$.

Let $Core_i$ be the set of statements that lie on the dependence paths from the scenario-relevant input variables to the corresponding output variables, defined as

$$Core_i = \bigcup_{x \in X_i} \bigcup_{a=1}^{r_x} \bigcup_{y \in Y_{D_i}} \bigcup_{b=1}^{k_y} \left(FS(P, (p_x^a, \{x\})) \cap BS(P, (l_y^b, \{y\})) \right).$$

The FSF-guided program slice for (T_i, D_i), denoted by $FPS(P, (T_i, D_i))$, is defined as the integration of

$$(DepClos_P(Core_i))\,\big|_{T_i}$$

and appropriate braces to make it an executable program. Here, $FS(P, (p_x^a, \{x\}))$ denotes the forward slice starting from the input variable x, and $BS(P, (l_y^b, \{y\}))$ denotes the backward slice with respect to the output variable y. $DepClos_P(Core_i)$ denotes the dependence closure of $Core_i$, which includes the data- and control-dependence predecessors, variable declarations, and other statements required to make the resulting slice executable and behavior-preserving. The notation $\big|_{T_i}$ denotes pruning branches that are proved to be infeasible under the testing condition T_i. When there is no ambiguity, $FPS(P, (T_i, D_i))$ is abbreviated as $FPS(T_i, D_i)$.

Based on the above definition, this subsection presents the construction algorithm for FSF-guided program slices. Given a program P and a functional scenario (T_i, D_i), Algorithm 1 first extracts the scenario-relevant input variables X_i and the output variables Y_{D_i} from (T_i, D_i). It then identifies their corresponding

input and export locations, and computes the core statements $Core_i$ by combining forward slices from the scenario-relevant input variables with backward slices from the corresponding output variables. After that, the algorithm computes the dependence closure $DepClos_P(Core_i)$, prunes branches that are infeasible under the testing condition T_i, and reconstructs the remaining statements into an executable scenario-specific program slice. The detailed procedure is summarized in Algorithm 1.

Algorithm 1 $FPS(P, (T_i, D_i))$: FSF-Guided Program Slicing

Require: A program P and a functional scenario (T_i, D_i)
Ensure: An executable FSF-guided program slice $FPS(P, (T_i, D_i))$

$X_{T_i} = ExtractInputVars(T_i)$
$X_{D_i} = ExtractInputVars(D_i)$
$Y_{D_i} = ExtractOutputVars(D_i)$
$X_i = X_{T_i} \cup X_{D_i}$ ▷ Scenario-relevant input variables
for all $x \in X_i$ **do**
  $IL_x = LocateInput(P, x)$ ▷ $IL_x = \{p_x^1, \cdots, p_x^{r_x}\}$
end for
for all $y \in Y_{D_i}$ **do**
  $EL_y = LocateExport(P, y)$ ▷ $EL_y = \{l_y^1, \cdots, l_y^{k_y}\}$
end for
$CFG = BuildCFG(P)$
$PDG = BuildPDG(P, CFG)$
$Core_i = \emptyset$
for all $x \in X_i$ **do**
  for all $p_x^a \in IL_x$ **do**
    for all $y \in Y_{D_i}$ **do**
      for all $l_y^b \in EL_y$ **do**
        $F = FS(P, (p_x^a, \{x\}))$ ▷ Forward slice from x
        $B = BS(P, (l_y^b, \{y\}))$ ▷ Backward slice with respect to y
        $Core_i = Core_i \cup (F \cap B)$
      end for
    end for
  end for
end for
$S_i = DepClos_P(Core_i)$ ▷ Add required dependence predecessors and declarations
for all branch br in S_i **do**
  if $T_i \wedge PathCond(br)$ is unsatisfiable **then**
    $S_i = Prune(S_i, br)$ ▷ Remove infeasible branch under T_i
  end if
end for
$FPS(P, (T_i, D_i)) = Reconstruct(S_i)$ ▷ Generate an executable program slice
return $FPS(P, (T_i, D_i))$

Consider the Java program shown in Listing 1. It implements a basic integer calculator function that performs addition, subtraction, multiplication, division, or modulo operations according to the given operator. For division and modulo operations, the program checks whether the second operand is nonzero before performing the computation. If division or modulo by zero occurs, or if the given operator is unsupported, the program raises an exception.

Example 4. For the calculator program shown in Listing 1, its functional scenario form can be described

as follows:

$$
\begin{aligned}
T_1 &:= \mathit{operator} = +, & D_1 &:= \mathit{return\_value} = \mathit{num1} + \mathit{num2},\\
T_2 &:= \mathit{operator} = -, & D_2 &:= \mathit{return\_value} = \mathit{num1} - \mathit{num2},\\
T_3 &:= \mathit{operator} = *, & D_3 &:= \mathit{return\_value} = \mathit{num1} \times \mathit{num2},\\
T_4 &:= \mathit{operator} = / \wedge \mathit{num2} \neq 0, & D_4 &:= \mathit{return\_value} = \mathit{num1}/\mathit{num2},\\
T_5 &:= \mathit{operator} = / \wedge \mathit{num2} = 0, & D_5 &:= \mathit{return\_value} = \text{ArithmeticException}_{\mathit{div0}},\\
T_6 &:= \mathit{operator} = \% \wedge \mathit{num2} \neq 0, & D_6 &:= \mathit{return\_value} = \mathit{num1} \bmod \mathit{num2},\\
T_7 &:= \mathit{operator} = \% \wedge \mathit{num2} = 0, & D_7 &:= \mathit{return\_value} = \text{ArithmeticException}_{\mathit{mod0}},\\
T_8 &:= \mathit{operator} \notin \{+, -, *, /, \%\}, & D_8 &:= \mathit{return\_value} = \text{IllegalArgumentException}_{\mathit{unsupported}}.
\end{aligned}
$$

Listing 1: Calculator Program

```
public class Calculator {
    public static int calculate(int num1, int num2, char operator) {
        int result;
        if (operator == '+') {
            result = num1 + num2;
        } else if (operator == '-') {
            result = num1 - num2;
        } else if (operator == '*') {
            result = num1 * num2;
        } else if (operator == '/' && num2 != 0) {
            result = num1 / num2;
        } else if (operator == '/' && num2 == 0) {
            throw new ArithmeticException("Division by zero");
        } else if (operator == '%' && num2 != 0) {
            result = num1 % num2;
        } else if (operator == '%' && num2 == 0) {
            throw new ArithmeticException("Modulo by zero");
        } else {
            throw new IllegalArgumentException("Unsupported operator");
        }
        return result;
    }
}
```

For illustration, we apply FSF-guided program slicing to the fourth functional scenario (T_4, D_4), which specifies the normal division behavior of the calculator program:

$$T_4 := \mathit{operator} = / \wedge \mathit{num2} \neq 0, \qquad D_4 := \mathit{return\_value} = \mathit{num1}/\mathit{num2}.$$

The testing condition T_4 restricts this functional scenario to inputs for which the selected operator is division and the second operand is nonzero. The defining condition D_4 specifies that the expected return value is the quotient of *num1* and *num2*. Accordingly, the input-variable sets extracted from T_4 and D_4 are

$$X_{T_4} = \{\mathit{operator}, \mathit{num2}\}, \qquad X_{D_4} = \{\mathit{num1}, \mathit{num2}\}.$$

Therefore, the set of scenario-relevant input variables is

$$X_4 = X_{T_4} \cup X_{D_4} = \{\mathit{operator}, \mathit{num1}, \mathit{num2}\}.$$

The output specified by D_4 is the return value of the method. Therefore, the corresponding output-variable set is

$$Y_{D_4} = \{\mathit{return\_value}\}.$$

In the original program, the observable output *return_value* is computed through the local variable *result* and exported by the statement `return result`. Thus, the return statement is identified as the export location associated with *return_value*.

According to the FSF-guided slicing algorithm, the slicing process first identifies the statements lying on the dependence paths from the input locations of the variables in X_4 to the export location associated with Y_{D_4}. The dependence closure of these statements is then computed to retain the variable declarations, data dependencies, and control dependencies required for constructing an executable program slice.

The testing condition T_4 is subsequently used to restrict the feasible execution paths. Under the constraint

$$operator = / \wedge num2 \neq 0,$$

the path conditions corresponding to addition, subtraction, multiplication, modulo, division by zero, and unsupported operators are unsatisfiable. Therefore, the statements belonging exclusively to these branches can be removed. The computation of the division result and the corresponding return statement are preserved.

After pruning the infeasible branches and reconstructing the retained statements, the scenario-specific program slice shown in Listing 2 is obtained.

Listing 2: The FSF-guided program slice for the functional scenario (T_4, D_4)

```
public class Calculator {
    public static int calculate(
             int num1, int num2, char operator) {
        int result;
        result = num1 / num2;
        return result;
    }
}
```

In the generated slice, T_4 is treated as the precondition that defines its valid input domain. Therefore, the condition $operator = / \wedge num2 \neq 0$ does not need to be evaluated again within the sliced program. Although the parameter *operator* no longer participates in the computation after the program has been specialized with respect to T_4, it is retained in the method signature to preserve interface compatibility with the original program.

For every input satisfying T_4, the generated slice returns

$$num1/num2,$$

which is identical to the result produced by the original program under the same functional scenario. Hence, the generated slice preserves the output behavior specified by D_4 over the input domain defined by T_4. The slice can subsequently be used as the executable program for testing-based formal verification of the functional soundness and functional completeness of the functional scenario (T_4, D_4).

The purpose of FSF-guided slicing is not merely to obtain a shorter program, but to remove computations and control-flow alternatives that are irrelevant to the selected functional scenario before performing TBFV. For (T_4, D_4), verification on the original calculator program still requires the preceding branch conditions to be evaluated and the control-flow structures associated with addition, subtraction, multiplication, division by zero, modulo, and unsupported operators to be processed, although these branches do not contribute to the behavior specified by D_4 under the constraint T_4. The generated slice eliminates these scenario-irrelevant statements and branches, thereby reducing the program size, number of executable statements, branch count, cyclomatic complexity, dependence-graph nodes, and path-exploration space. Consequently, fewer execution paths and path constraints need to be processed during TBFV, which can reduce the number of testing iterations, the workload of the Z3 solver, and the overall time required to verify functional soundness and functional completeness. A detailed empirical evaluation of these reductions, including their effects on program size and verification cost, is presented in Section 6.

5.3. Testing Stage

After applying FSF-guided program slicing to each functional scenario (T_i, D_i), we obtain a scenario-specific program slice. In the testing stage, test cases are generated and executed on the slice to collect the traversed paths and derive the corresponding path conditions and state representations. The detailed steps are depicted in Fig. 2 and explained as follows.

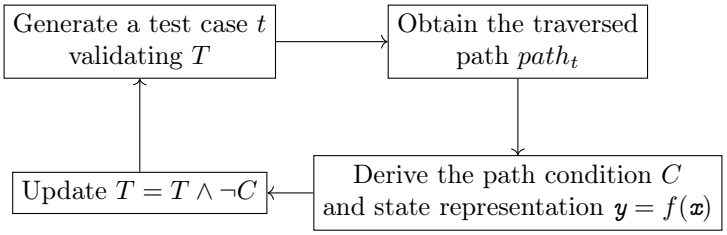

Figure 2: Testing stage

- Step 1: Generate a test case t validating T.
- Step 2: Execute the program slice with the test case t and record the execution path, consisting of assignment statements, judgments of conditions, and other statements which do not change the program state (e.g. print statements).
- Step 3: Derive the path condition C and state representation $\boldsymbol{y} = f(\boldsymbol{x})$ according to the following axioms.

$$\frac{}{\{Q(E/x)\}\ x = E\ \{Q\}} \quad \text{(Assignment)}$$

$$\frac{}{\{Q\}\ S\ \{Q\}} \quad \text{(Identity)}$$

$$\frac{}{\{B \wedge Q\}\ B\ \{Q\}} \quad \text{(Branch-T)}$$

$$\frac{}{\{\neg B \wedge Q\}\ \neg B\ \{Q\}} \quad \text{(Branch-F)}$$

$$\frac{\{Q_1\}\ S_1\ \{Q_2\},\ \{Q_2\}\ S_2\ \{Q_3\}}{\{Q_1\}\ S_1; S_2\ \{Q_3\}} \quad \text{(Composition)}$$

  The Assignment axiom states that the assignment $x = E$ correctly assigns the result of evaluating expression E to variable x with respect to the given post-condition Q and the pre-condition $Q(E/x)$, a predicate resulting from substituting E for all of the free occurrences of x in Q. Note that if x does not appear in Q, $Q(E/x)$ will be equal to Q. The Identity axiom is applied to the statements that do not change the program state, such as *printing statement* and *break statement*. It describes that the pre-condition and post-condition for any of such statements are the same because none of them changes the state. A judgement statement B is a Boolean expression that may be used in $\textbf{if}(B)\{\cdots\}$ or $\textbf{while}(B)\{\cdots\}$. The Branch-T and Branch-F axioms correspond to the two situations whether the branch is valid or not, respectively. The Composition axiom allows for the sequential composition of program fragments. Finally, C is a constraint on input variables $x_1, \cdots, x_n$ and $\boldsymbol{y} = f(\boldsymbol{x})$ records a substitution where the output variable $y_1, \cdots, y_m$ are expressions of $x_1, \cdots, x_n$.
- Step 4: Since all test cases validating C will induce the same execution path, to generate any such test case makes no sense. Therefore, we update T as $T \wedge \neg C$ and repeat Step 1.

Example 5. Consider the Java program which calculates and outputs the smallest integer n such that the sum of the cubes of natural numbers from 1 to n is greater than or equal to the user-inputted integer x.

Moreover, if x is non-positive, the output should be $error = -1$. The corresponding functional scenario form consists of two functional scenarios, (T_1, D_1) and (T_2, D_2), defined as follows:

$$T_1 := x \leq 0, D_1 := error = -1$$
$$T_2 := x > 0, D_2 := \frac{(n-1)^2 n^2}{4} < x \leq \frac{n^2(n+1)^2}{4}$$

Listing 3: UserInputProgram

```
public class UserInputProgram {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int x = scanner.nextInt();
        if (x <= 0) {
            int error = -1;
            System.out.println("error = " + error);
        } else {
            int sum = 0;
            int n = 0;
            while (sum < x) {
                n = n + 1;
                sum = sum + (n * n * n);
            }
            System.out.println("n = " + n);
        }
    }
}
```

Example 6. Continuing Example 5, for the program slice $FPS(P, (T_2, D_2))$, the first case is generated randomly from the testing condition $x > 0$ and assume that it is 6. Then, we will obtain the following execution path and the corresponding backward derivation process.

$\{1 < x \leq 9, n = 2\}$
$\neg$(x<=0)
$\{0 < x \wedge 1 < x \wedge \neg 9 < x, \hat{n} = 2\}$
sum=0
$\{sum < x \wedge sum + 1 < x \wedge \neg sum + 9 < x, \hat{n} = 2\}$
n=0
$\{sum < x \wedge sum + (n+1)^3 < x \wedge \neg sum + (n+1)^3 + (n+2)^3, \hat{n} = n + 2\}$
sum<x
$\{sum + (n+1)^3 \wedge \neg sum + (n+1)^3 + (n+2)^3, \hat{n} = n + 1 + 1)\}$
n=n+1
$\{sum + n^3 \wedge \neg sum + n^3 + (n+1)^3, \hat{n} = n + 1\}$
sum=sum+(n*n*n)
$\{sum < x \wedge \neg sum + (n+1)^3 < x, \hat{n} = n + 1\}$
sum<x
$\{\neg sum + (n+1)^3 < x, \hat{n} = n + 1\}$
n=n+1

$\{\neg sum + n^3 < x, \hat{n} = n\}$

`sum=sum+(n*n*n)`

$\{\neg sum < x, \hat{n} = n\}$

`¬sum<x`

$\{\hat{n} = n\}$

In the state representation $\hat{n} = n$ (the start point of the derivation process), $\hat{n}$ represents the final value of the output variable n and thus it could not be substituted in the derivation process while n (on the right side) should be substituted according to the axioms. At the end of the derivation process, the hat of $\hat{n}$ will be removed. Finally, we obtain the path condition $1 < x \leq 9$, representing all inputs inducing the same path, and the corresponding state representation $n = 2$, representing the corresponding output value of n is 2 for this path. For brevity, the statements satisfying the Identity axiom are ignored. Then, a new test case will be randomly generated from the expression $x > 0 \wedge \neg 1 < x \leq 9$. Assume it is 120, the corresponding path condition and state representation will be $100 < x \leq 225$ and $n = 5$. Repeat these processes to obtain more test cases.

5.4. Verification Stage

Since the steps are carried out iteratively in Fig. 2, a natural question is when the process should be stopped. An intuitive termination is achieved if the finally updated T is a contradiction, which means that all execution paths of all test cases validating T have been generated. In this case, functional soundness and functional completeness are two decidable issues, as stated in Theorem 2.

Theorem 2. *Given a functional scenario (T, D) and a program P, let $S = FPS(P, (T, D))$ be the FSF-guided program slice. Assume that, for S, the generated test cases are $t_1, \cdots, t_k$ with the corresponding $C_1 \wedge \boldsymbol{y} = f_1(\boldsymbol{x}), \cdots, C_k \wedge \boldsymbol{y} = f_k(\boldsymbol{x})$ such that $T \Rightarrow C_1 \vee \cdots \vee C_k$ is a tautology.*

Then, the following hold:

- *(T, D) is sound with regard to S if and only if $\bigwedge_{i=1}^{k}(T \wedge C_i \Rightarrow D(f_i(\boldsymbol{x})/\boldsymbol{y}))$ is a tautology;*
- *(T, D) is complete with regard to S if and only if $\exists \boldsymbol{x}\,(T \wedge D) \Rightarrow \exists i \exists \boldsymbol{x}\,(T \wedge C_i \wedge \boldsymbol{y} = f_i(\boldsymbol{x}))$ is a tautology.*

In some cases, to make $T \Rightarrow C_1 \vee \cdots \vee C_k$ a tautology may need a huge k, so we may stop early and obtain weaker notions, as in Definition 11. Note that the early stop may depend on the value of k or some coverage criterion, which will be studied in the future.

Definition 11. Given a functional scenario (T, D) and a program P, let $S = FPS(P, (T, D))$ be the FSF-guided program slice. Assume that, for S, the generated test cases are $t_1, \cdots, t_k$ with the corresponding $C_1 \wedge \mathtt{y} = f_1(\mathtt{x}), \cdots, C_k \wedge \mathtt{y} = f_k(\mathtt{x})$. Then, we say that

- (T, D) is locally sound with regard to S on $\bigvee_{i=1}^{k} C_i$ if $\bigwedge_{i=1}^{k}(T \wedge C_i \Rightarrow D(f_i(\mathtt{x})/\mathtt{y}))$ is a tautology;
- (T, D) is locally complete with regard to S on $\bigvee_{i=1}^{k} C_i$ if $\exists \mathtt{x}\,(T \wedge (\bigvee_{i=1}^{k} C_i) \wedge D) \Rightarrow \exists i \exists \mathtt{x}\,(T \wedge C_i \wedge \mathtt{y} = f_i(\mathtt{x}))$ is a tautology.

6. Experiment

6.1. Experiment Setup

Although our methodology is a universal technique, we develop tool support for the Java language to evaluate its efficiency and accuracy. In the extended version, the tool further supports FSF-guided program slicing, which constructs a scenario-specific executable slice for each functional scenario before the testing and verification stages. The tool consists of modules for code parsing, FSF parsing, FSF-guided slicing, test case generation, dynamic execution, iterative execution, and a verification engine. The workflow is as follows: After users input the target code and the corresponding functional scenario, the tool automatically parses the

program and the scenario, constructs the FSF-guided program slice, generates constraint-based test cases on the slice, executes the sliced program, obtains the corresponding path conditions and state representations, and verifies the functional soundness and completeness of the program using the Z3 solver. The tool is developed on IntelliJ IDEA 2026.1.1 and is deployed in a Linux cluster environment (Ubuntu 26.04 LTS). The core logic of the tool consists of 1650 lines of Java code, while an auxiliary module, comprising 700 lines of code, handles dynamic execution and verification functionalities. Key dependencies include Z3 (4.12.2), JavaParser (3.25.4), and Subprocess (3.10.9), which together enable automatic parsing, slicing, execution, and verification of Java code within the tool.

The experiment recruited 10 graduate students and PhD students with intermediate Java programming skills (1–3 years of development experience) to manually analyze 250 Java programs and derive the corresponding functional scenarios. The programs were collected from diverse sources, including the TBFV4J-Dataset, SpecGenBench, the LeetCode platform, and real-world system code from automotive driver-assistance and UAV flight-control systems. The dataset also includes programs for typical computational problems, such as cube sum calculation and quadratic equation solving. The authors then used the tool to automatically verify the functional soundness and functional completeness of the derived functional scenarios.

Relation to the conference-version experiment. The conference version included a preliminary experiment involving 20 functional scenarios manually derived by students. That experiment mainly demonstrated the feasibility of using TBFV to identify functionally unsound or incomplete scenario specifications. The present journal extension does not repeat the case-level results of that pilot study. Instead, it conducts a substantially larger scenario-level evaluation involving 250 Java programs and 740 functional scenarios. The extended evaluation systematically investigates program-scale reduction, TBFV cost reduction, and preservation of verification results through RQ1–RQ3.

6.2. An Illustrative Example

Continuing Examples 4 and 6, after the program P and the functional scenario (T, D) were given to the tool, the tool generated the corresponding slice $FPS(P, (T_1, D_1))$, only one test case ($x = -10$ in the experiment) was generated and the updated testing condition is a contradiction in the second iteration, since all test cases $x \leq 0$ induce the same path. The tool execution time was 0.980829 seconds. When considering the second functional scenario, we replaced $T_2 : x > 0$ with $0 < x \leq 500$ as the initial testing condition to avoid the endless generation of test cases. Upon completion of program execution, 7 test cases were generated within the interval $(0, 500]$, in order: $x = 4$ (corresponding to $n = 2$), $x = 365$ (corresponding to $n = 6$), $x = 58$ (corresponding to $n = 4$), $x = 1$ (corresponding to $n = 1$), $x = 470$ (corresponding to $n = 7$), $x = 215$ (corresponding to $n = 5$), $x = 23$ (corresponding to $n = 3$). The tool execution time was 7.205893 seconds. In summary, there were 10 iterations and the total execution time was 8.186722 seconds. Therefore, we can calculate an average iteration execution time as $8.186722/10 = 0.818672$ seconds. In the experiment, the average time is used as the efficiency index.

Table 1: Verification Results of D_2 and Its Mutants

No.	D_2	Soundness	Completeness
1	$\frac{(n-1)^2n^2}{4} < x \leq \frac{n^2(n+1)^2}{4}$	Sound	Complete
2	$\frac{(n-1)^2n^2}{4} \leq x \leq \frac{n^2(n+1)^2}{4}$	Sound	Incomplete
3	$\frac{(n-1)^2n^2}{4} < x < \frac{n^2(n+1)^2}{4}$	Unsound	Complete
4	$\frac{(n-1)^2n^2}{4} \leq x < \frac{n^2(n+1)^2}{4}$	Unsound	Incomplete

To evaluate the accuracy of our methodology, we made three mutants of D_2 and the verification results are shown in Table 1. The experimental results show that the original functional scenario is both sound and complete with regard to the program. It is worth noting that when the lower bound constraint of D_2 is relaxed (No.2), the program still maintains functional soundness, but due to the lack of coverage for some values of

Table 2: Reduction of program scale by FSF-guided slicing

Category	Programs	Tasks	Avg. FSF	LOC Ratio	Exec. Stmt. Ratio	CC Ratio
Branched	100	339	3.39	46.90%	35.46%	34.78%
Sequential	63	137	2.17	99.30%	100.00%	100.00%
Single-path-Loop	21	57	2.71	73.40%	59.98%	70.00%
Multi-path-Loop	31	104	3.35	57.80%	50.00%	55.53%
Nested-Loop	35	103	2.94	51.30%	48.75%	47.02%
Overall	250	740	2.96	60.79%	53.19%	54.19%

output variables, completeness fails. Conversely, when the upper bound constraint is strengthened (No.3), the program maintains output completeness, but due to incorrect functional implementation, soundness fails. It should be mentioned that the test cases generated for No.2, No.3 and No.4 were mutually distinct, as well as different from the previous values, due to the random generation, but the amounts were still 7 since the 7 traversed paths have covered the condition $0 < x \leq 500$ and the execution times were very close.

6.3. Experiment Results

RQ1. Does FSF-guided slicing reduce program scale?

RQ1 evaluates whether the proposed FSF-guided slicing technique can effectively reduce the static scale of Java programs. Since FSF-guided slicing constructs a scenario-specific slice for each functional scenario, we use $(P, (T_i, D_i))$ as the basic experimental unit, where P is an original Java program and (T_i, D_i) is one functional scenario in its FSF specification. We classify the original Java programs into five structural categories: Branched, Sequential, Single-path-Loop, Multi-path-Loop, and Nested-Loop. As shown in Table 2, the 250 Java programs contain 740 functional scenarios in total, and thus correspond to 740 FSF-guided slicing tasks.

We use three metrics to measure program scale: lines of code (LOC), number of executable statements, and cyclomatic complexity (CC). The number of executable statements refers to the number of statically existing executable statements in the program, rather than the number of statements dynamically executed during testing. For each slicing task, the LOC, executable-statement, and cyclomatic-complexity ratios are calculated by dividing the corresponding metric of the generated slice by that of the original program. For a metric M, the ratio of the j-th slicing task in category c is defined as

$$R_{c,j}^{M} = \frac{M(S_{c,j})}{M(P_{c,j})} \times 100\%,$$

where $P_{c,j}$ is the original program involved in the j-th slicing task of category c, and $S_{c,j}$ is the generated slice. The category-level ratio is computed as

$$R_{c}^{M} = \frac{1}{N_c} \sum_{j=1}^{N_c} \frac{M(S_{c,j})}{M(P_{c,j})} \times 100\%,$$

where N_c is the number of valid slicing tasks in category c. The overall ratio is computed as a weighted average over all slicing tasks:

$$R_{\text{overall}}^{M} = \frac{\sum_c N_c R_c^M}{\sum_c N_c}.$$

Therefore, the overall values in Table 2 are weighted by the number of slicing tasks in each category, rather than by directly averaging the five category-level values.

The results show that FSF-guided slicing can substantially reduce program scale. Overall, the generated slices retain 60.79% of the original LOC, 53.19% of the original executable statements, and 54.19% of the original cyclomatic complexity. Equivalently, the technique removes 39.21% of LOC, 46.81% of executable statements, and 45.81% of cyclomatic complexity on average.

The reduction effect varies across program structures. Branched programs achieve the most significant reduction, retaining only 46.90% of LOC, 35.46% of executable statements, and 34.78% of cyclomatic complexity. This result is consistent with the design of FSF-guided slicing, because the testing condition T_i can identify scenario-specific feasible branches and remove branches irrelevant to the current functional scenario, as illustrated in Example 4. Sequential programs, in contrast, retain almost the entire original program, with 99.30% of LOC and 100.00% of executable statements and cyclomatic complexity. This is also expected because sequential programs contain few or no alternative control-flow branches to be pruned. Loop-related programs show moderate reductions. Single-path-Loop programs retain 73.40% of LOC and 59.98% of executable statements, while Multi-path-Loop and Nested-Loop programs retain lower ratios, indicating that FSF-guided slicing is more effective when loop structures are combined with scenario-dependent control flow.

Therefore, the answer to RQ1 is positive. FSF-guided slicing effectively reduces the static scale and structural complexity of programs, especially for branch-intensive and complex-control-flow programs. The results also show that the technique behaves reasonably across different categories: it produces substantial reductions when scenario-specific pruning is possible, while preserving most of the code for sequential programs where little irrelevant control-flow structure exists.

RQ2. Does FSF-guided slicing reduce the cost of TBFV?

RQ2 evaluates whether FSF-guided slicing can reduce the cost of testing-based formal verification. While RQ1 focuses on the reduction of static program scale, RQ2 further investigates whether such scenario-specific slices can reduce the dynamic verification cost incurred by TBFV. This question is important because the purpose of FSF-guided slicing is not merely to obtain smaller programs, but to make subsequent testing-based formal verification more efficient. In particular, we examine whether running TBFV on $FPS(P, (T_i, D_i))$ is cheaper than running it on the original program P for the same functional scenario (T_i, D_i).

For each scenario-specific slicing task, we run TBFV on both the original program and the corresponding FSF-guided slice under the same testing condition, input range, solver configuration, and stopping condition. To reduce measurement fluctuation, each experiment is repeated ten times, and the average time is used for comparison. We measure four cost indicators: dynamic execution time, path derivation time, SMT solving time, and total time. Dynamic execution time refers to the time spent executing the program and collecting execution information. Path derivation time refers to the time spent deriving path conditions and state representations. SMT solving time refers to the time consumed by the SMT solver. Total time refers to the end-to-end time, including slicing, execution, path derivation, SMT solving, and result generation.

For a cost metric C, the cost ratio of the j-th slicing task in category c is calculated as

$$R^C_{c,j} = \frac{\overline{C}(FPS(P_{c,j}, (T_i, D_i)))}{\overline{C}(P_{c,j}, (T_i, D_i))},$$

where $\overline{C}$ denotes the average value over ten repeated runs. A lower ratio indicates a larger reduction in TBFV cost. The category-level ratio is computed as

$$R^C_c = \frac{1}{N_c} \sum_{j=1}^{N_c} R^C_{c,j},$$

where N_c is the number of valid scenario-specific verification tasks in category c. The overall ratio is computed as a weighted average over all slicing tasks:

$$R^C_{\text{overall}} = \frac{\sum_c N_c R^C_c}{\sum_c N_c}.$$

Therefore, the overall results are weighted by the number of scenario-specific slicing tasks in each category.

Table 3 shows that FSF-guided slicing reduces the cost of TBFV across all measured indicators. Overall, the dynamic execution time is reduced to 0.767 of the original cost, the path derivation time to 0.795, the SMT solving time to 0.750, and the total time to 0.775. In other words, FSF-guided slicing reduces the total

Table 3: Reduction of TBFV cost by FSF-guided slicing

Category	Dataset		Cost ratio (slice/original)			
	Programs	Tasks	Dynamic Exec.	Path Deriv.	SMT Solv.	Total Time
Branched	100	339	0.653	0.721	0.660	0.682
Sequential	63	137	0.943	0.982	0.964	0.971
Single-path-Loop	21	57	0.893	0.849	0.863	0.874
Multi-path-Loop	31	104	0.742	0.750	0.745	0.746
Nested-Loop	35	103	0.865	0.806	0.702	0.794
Overall	250	740	0.767	0.795	0.750	0.775

Each ratio is computed as the cost on the FSF-guided slice divided by the cost on the original program. Each time value is averaged over ten repeated runs. The overall ratios are weighted by the number of scenario-specific slicing tasks.

TBFV cost by 22.5% on average, even when the slicing time itself is included in the total time. This result indicates that the reduced scenario-specific slices can effectively lower the end-to-end verification cost.

The reduction effect varies across program structures. Branched programs obtain the largest overall reduction, with a total time ratio of 0.682. This result is consistent with the design of FSF-guided slicing: the testing condition T_i can identify feasible branches for a specific functional scenario and remove irrelevant branches, thereby reducing the number and complexity of paths to be analyzed. Multi-path-Loop and Nested-Loop programs also show clear reductions, with total time ratios of 0.746 and 0.794, respectively. For these programs, slicing can remove scenario-irrelevant branches inside or around loop structures, which helps simplify path derivation and SMT solving. Single-path-Loop programs show a more moderate reduction, with a total time ratio of 0.874, because the loop structure itself usually needs to be preserved for the corresponding functional behavior.

For Sequential programs, the reduction is limited, with a total time ratio of 0.971. This is also reasonable. Since sequential programs usually contain few or no explicit control-flow branches, FSF-guided slicing cannot significantly reduce the number of explored paths. However, a slight reduction can still be observed because the slice may simplify the expressions and variables involved in each path. For example, even without explicit `if`/`else` branches, sequential programs may contain Boolean expressions, arithmetic expressions, conditional expressions, or other computations that affect path-condition derivation and SMT solving. Therefore, the benefit for Sequential programs mainly comes from simplifying scenario-specific symbolic representations rather than reducing control-flow paths.

It is worth noting that the reduction in verification time does not exactly match the reduction in static program size reported in RQ1. Static metrics such as LOC, executable statements, and cyclomatic complexity measure the structural size of a program, while TBFV cost is determined by dynamic execution behavior, explored paths, path-condition construction, SMT formula complexity, solver difficulty, and framework overhead. Therefore, FSF-guided slicing may yield different reduction ratios for static scale and verification time. Static reduction explains why verification can become cheaper, but it does not linearly determine the actual time reduction.

Therefore, the answer to RQ2 is positive. FSF-guided slicing reduces the cost of TBFV, especially for programs with scenario-dependent branches and complex control-flow structures. The results confirm that the proposed slicing technique not only reduces static program scale, but also improves the efficiency of subsequent testing-based formal verification.

RQ3. Does FSF-guided slicing preserve the verification results?

RQ3 evaluates whether FSF-guided slicing preserves the verification results produced by testing-based formal verification. This question is essential because FSF-guided slicing is used as a preprocessing step before TBFV. Although RQ1 and RQ2 show that the proposed slicing technique can reduce program scale and verification cost, respectively, such reductions are meaningful only if the generated slices do not change the final verification judgments. Therefore, RQ3 focuses on whether TBFV produces consistent functional soundness and functional completeness results before and after applying FSF-guided slicing.

The generated slice is expected to preserve the verification-relevant behavior of the original program

with respect to the same functional scenario (T_i, D_i). Before and after slicing, the FSF scenario remains unchanged; only the analyzed program changes from the original program P to the scenario-specific slice $FPS(P, (T_i, D_i))$. Since T_i defines the input space to be considered and D_i specifies the output behavior to be verified, if P and $FPS(P, (T_i, D_i))$ produce the same values for the output variables involved in D_i under inputs satisfying T_i, then the functional soundness and functional completeness judgments should be preserved. Therefore, in RQ3, we evaluate whether TBFV produces consistent verification results before and after applying FSF-guided slicing.

For each scenario-specific verification task $(P, (T_i, D_i))$, we run TBFV on both the original program P and the corresponding FSF-guided slice $FPS(P, (T_i, D_i))$ under the same testing condition, input range, solver configuration, and stopping condition. A task is regarded as definite when both the original program and the corresponding slice produce definite TBFV judgments. A task is regarded as inconclusive when both sides fail to produce definite judgments. In our experiment, no mixed-availability case was observed, i.e., there is no task in which only one side produces a definite judgment. The agreement calculation is conducted only on definite tasks, because only these tasks provide valid verification judgments that can be compared.

Let

$$V_P(T_i, D_i) = (S_P(T_i, D_i), C_P(T_i, D_i))$$

denote the verification result of the original program, where $S_P(T_i, D_i)$ and $C_P(T_i, D_i)$ are the functional soundness and functional completeness results, respectively. Similarly, let

$$V_{FPS}(T_i, D_i) = (S_{FPS}(T_i, D_i), C_{FPS}(T_i, D_i))$$

denote the verification result of the FSF-guided slice. A definite verification task is considered result-preserving if

$$V_P(T_i, D_i) = V_{FPS}(T_i, D_i).$$

In other words, both the soundness judgment and the completeness judgment must be identical before and after slicing.

We use two types of metrics to evaluate verification-result preservation. The first metric is conclusive preservation, which is computed over all scenario-specific verification tasks. It measures the proportion of tasks for which both sides produce definite and identical verification judgments:

$$\textit{Conclusive Preservation} = \frac{|\{j \mid V_P^j = V_{FPS}^j \text{ and both sides are definite}\}|}{N} \times 100\%,$$

where N is the total number of scenario-specific verification tasks.

The second type of metric is agreement, which is computed only over definite tasks. The soundness agreement measures whether the soundness judgment, including both Sound and Unsound results, is preserved after slicing. The completeness agreement measures whether the completeness judgment, including both Complete and Incomplete results, is preserved after slicing. For a set of N_d definite tasks, the agreement rate is calculated as

$$\textit{Agreement} = \frac{|\{j \mid R_P^j = R_{FPS}^j\}|}{N_d} \times 100\%,$$

where R_P^j and R_{FPS}^j denote the corresponding soundness or completeness judgments of the original program and the FSF-guided slice, respectively.

Table 4 reports the preservation results. Def. and Inconcl. denote tasks where both sides produced definite judgments and tasks where both sides were inconclusive, respectively. No mixed-availability case was observed. Conclusive preservation is computed over all tasks as the number of tasks for which both sides produced definite and identical judgments divided by the total number of tasks. Agreement is computed only over definite tasks and covers both possible outcomes of each judgment, i.e., Sound/Unsound for soundness and Complete/Incomplete for completeness.

Among the 740 scenario-specific verification tasks, 718 tasks produced definite TBFV judgments on both the original programs and their corresponding FSF-guided slices, while 22 tasks were inconclusive on both

Table 4: Preservation of TBFV results by FSF-guided slicing

Category	Dataset		Availability		Conclusive pres. (%)	Agreement (%)	
	Prog.	Tasks	Def.	Inconcl.		Sound. judg.	Compl. judg.
Branched	100	339	333	6	98.23	100.00	100.00
Sequential	63	137	133	4	97.08	100.00	100.00
Single-path-Loop	21	57	53	4	92.98	100.00	100.00
Multi-path-Loop	31	104	104	0	100.00	100.00	100.00
Nested-Loop	35	103	95	8	92.23	100.00	100.00
Overall	250	740	718	22	97.03	100.00	100.00

sides. No mixed-availability case was observed. Since all 718 definite tasks produced identical soundness and completeness judgments before and after slicing, the overall conclusive preservation rate is 97.03%. Moreover, on the definite tasks, both the soundness agreement and the completeness agreement reach 100% in all five program categories. This means that whenever TBFV produces definite judgments on both the original program and the corresponding slice, FSF-guided slicing does not change the functional soundness or functional completeness judgment.

For the 22 inconclusive tasks, we further conducted a post-hoc analysis of the error cases. The analysis shows that the root cause of these tasks lies in manually written FSF specifications that do not strictly follow the required FSF format, or that contain illegal or unsupported symbols. When such FSF specifications are used as inputs to the tool, they may trigger failures in the subsequent analysis process and prevent the tool from producing final verification judgments. For example, malformed or unsupported FSF expressions may cause failures in path-condition construction, incorrect substitution of defining conditions, failures in translating Java expressions into Z3 expressions, or tool exceptions caused by unsupported expressions. Therefore, these inconclusive tasks are caused by invalid or unsupported FSF inputs rather than by the FSF-guided slicing technique itself, and they are not regarded as inconsistent cases.

The 100% agreement on definite tasks is consistent with the design of FSF-guided slicing. The slice preserves the dependence paths from scenario-relevant input variables to the output variables specified by the defining condition D_i, while the testing condition T_i is used to prune branches that are infeasible for the current functional scenario. Therefore, the generated slice is not required to preserve the entire behavior of the original program. Instead, it is expected to preserve the verification-relevant behavior with respect to the given functional scenario (T_i, D_i).

Therefore, the answer to RQ3 is positive on all definite tasks. FSF-guided slicing preserves the TBFV verification results for functional scenarios: the functional soundness and functional completeness judgments obtained from the generated slices are consistent with those obtained from the original programs. This result supports the reliability of using FSF-guided slicing as a preprocessing technique for TBFV.

7. Threats to Validity

This section discusses the main threats to the validity of our empirical evaluation.

Internal validity. The threats to internal validity mainly come from two aspects. The first threat lies in the implementation of the proposed tool chain. Our tool integrates several components, including FSF-guided slicing, test generation, path-condition derivation, state-representation construction, and SMT solving. Errors in any of these components may affect the experimental results. To reduce this threat, we used the same tool configuration, input range, solver configuration, and stopping condition in the repeated experiments and when comparing the original programs with their corresponding FSF-guided slices.

The second threat comes from manually written FSF specifications. The functional scenarios used in the experiment were written manually. Although we provided many validated FSF examples before the experiment, some failed tasks were still caused by FSF specifications that did not strictly follow the required format, or that contained illegal or unsupported symbols. Such inputs may prevent the tool from correctly recognizing and processing the functional scenarios, leading to failures in path-condition construction, defining-condition substitution, Java-to-Z3 expression translation, or unsupported-expression

handling. Therefore, these failed tasks were treated as inconclusive tasks rather than inconsistent tasks. For RQ3, a task was used for agreement calculation only when both the original program and the corresponding slice produced definite TBFV results. This treatment avoids attributing failures caused by invalid FSF inputs to the slicing technique itself. It also indicates that the current tool requires stronger checking and normalization of FSF specifications.

Construct validity. The threats to construct validity concern whether the selected metrics properly measure the intended effects. For RQ1, we used lines of code, the number of executable statements, and cyclomatic complexity to measure program-scale reduction. These metrics can reflect the static size and structural complexity of programs, but they cannot fully characterize all aspects of program semantics or verification difficulty. For RQ2, we measured dynamic execution time, path-derivation time, SMT-solving time, and total verification time. These metrics directly reflect the cost of TBFV, but the total time may also be affected by fixed framework overhead, JVM execution, and system-level fluctuations. Since the execution time of each tool component may slightly vary across different runs, each verification task in RQ2 was repeatedly executed ten times, and the average time was used to reduce the influence of random runtime fluctuations.

External validity. The threats to external validity concern the generalizability of the results. The experiment was conducted on 250 Java programs and 740 scenario-specific verification tasks. These programs cover several structural categories, including branched programs, sequential programs, single-path loops, multi-path loops, and nested loops. Although this classification helps evaluate the proposed method on different control-flow structures, the benchmark programs may not fully represent large-scale industrial software systems. In particular, the current implementation still has limited support for complex data structures, complex library calls, object-oriented features, and method invocations.

In addition, all experiments were conducted under a specific implementation environment and a specific SMT solver configuration. Different machines, hardware platforms, solver versions, and input ranges may lead to different execution times and verification costs. Nevertheless, since the original programs and their corresponding slices were compared under the same experimental settings, the relative reduction results reported in this paper still provide useful evidence for the effectiveness of FSF-guided slicing.

8. Related Work

Testing-based formal verification (TBFV) combines specification-based testing with formal reasoning over test-induced program paths. Early studies execute concrete test cases and use Hoare logic to establish path correctness, with support from program inspection and prototype tools Liu (2012); Liu and Nakajima (2013); Liu (2016). Later studies replace Hoare-logic reasoning with symbolic execution, introduce coverage and fault-localization support, and extend TBFV to operations involving software packages and neural networks Wang and Liu (2018, 2019); Wang et al. (2020); Liu and Liu (2023); Liu et al. (2022, 2024). These studies improve the automation, applicability, and diagnostic ability of TBFV, but they generally perform verification on the original program or on directly exercised program paths.

Functional Scenario Form (FSF) decomposes a formal operation specification into a disjunction of functional scenarios. Each scenario (T_i, D_i) consists of a testing condition T_i, which characterizes an applicable input region, and a defining condition D_i, which specifies the expected input–output relation. Existing studies transform formal specifications into FSFs, select functional scenarios for specification animation, and generate test cases and test oracles from them Liu et al. (2010); Liu (2015); Liu and Nakajima (2022). In these studies, FSF mainly serves as an artifact for specification decomposition, scenario selection, and test generation. Its role in guiding program reduction before TBFV has not been sufficiently explored.

Program slicing extracts statements relevant to a slicing criterion, which is conventionally defined by a program point and a set of variables Weiser (1984). Recent work has improved practical slicing for Java programs and exception-handling constructs Galindo et al. (2023b,a). Conditioned slicing restricts the preserved behavior by using a predicate on the initial state, while specification-based slicing uses a precondition–postcondition pair as the slicing criterion Canfora et al. (1998); Lee et al. (2001). Program slicing has also been evaluated as a preprocessing technique for software verification, where removing property-irrelevant

code can improve reachability analysis and verification efficiency Chalupa and Strejcek (2019); Chai et al. (2024). However, these approaches usually focus on program points, variables, assertions, reachability properties, or general pre/postconditions, rather than the structure and verification objective of an FSF scenario.

Our previous conference work integrated TBFV with output-oriented slicing and reported a preliminary evaluation Liu et al. (2025). The present work differs from both conventional slicing and traditional TBFV. FSF-guided slicing takes the whole functional scenario (T_i, D_i) as its slicing criterion: T_i identifies the applicable input region and supports infeasible-branch pruning, while D_i identifies the output behavior that should be preserved. Based on this criterion, the proposed technique preserves the data and control dependencies from scenario-relevant inputs to the outputs specified by D_i, and reconstructs an executable slice for each functional scenario. Unlike traditional TBFV, which analyzes the original program paths, our method performs TBFV on scenario-specific slices. Therefore, the preservation target is not the complete behavior of the original program, but the verification-relevant behavior with respect to (T_i, D_i). This design enables reductions in program scale and verification cost while preserving the functional soundness and functional completeness results required by TBFV.

9. Conclusion

In this paper, we introduce the notions of functional soundness and functional completeness based on the notion of functional scenario. We develop a general methodology to verify the functional soundness and completeness of a functional scenario by integrating FSF-guided program slicing and testing-based formal verification. In the methodology, FSF-guided program slicing is used to construct scenario-specific executable slices for functional scenarios, thereby reducing the program scale and simplifying the subsequent verification task. Testing-based formal verification is then applied to avoid the difficulty of manually deriving loop invariants and to enhance the automation of the verification process. We also implement tool support for the Java language and conduct an empirical experiment to evaluate the proposed methodology.

The experimental results show that FSF-guided slicing is effective in reducing both program scale and verification cost. For RQ1, the generated slices retain, on average, 60.79% of the original lines of code, 53.19% of the original executable statements, and 54.19% of the original cyclomatic complexity. The reduction is particularly significant for branched and complex-control-flow programs, where the testing condition of a functional scenario can help identify feasible branches and remove scenario-irrelevant code. For RQ2, FSF-guided slicing reduces the overall cost of TBFV to 77.5% of the original cost, even when the slicing time is included in the total time. The results indicate that scenario-specific slicing not only reduces static program scale, but also lowers the dynamic cost of program execution, path derivation, SMT solving, and result generation during TBFV.

The results of RQ3 further demonstrate the reliability of FSF-guided slicing as a preprocessing technique for TBFV. Among 740 scenario-specific TBFV tasks, 718 tasks successfully produced comparable verification results on both the original programs and their corresponding slices. On these comparable tasks, the functional soundness and functional completeness judgments are fully consistent before and after slicing. This shows that FSF-guided slicing preserves the verification-relevant behavior with respect to each functional scenario, rather than merely producing smaller programs. Therefore, the proposed technique can improve the efficiency of TBFV while maintaining the correctness of the final verification judgments on comparable cases.

There are several topics worth exploring further. For instance, the current methodology should be extended to better support programs with complex data structures, method invocations, and richer Java language features, since the current axioms and expression translation rules cannot fully handle such cases. In addition, the robustness of the tool chain should be further improved to reduce inconclusive cases caused by path-condition construction failures, expression-to-SMT translation failures, and unsupported expressions. Addressing these issues will further enhance the applicability and reliability of the proposed methodology.

Declaration of Competing Interest

The authors declare no known competing financial interests or personal relationships that could have influenced the work reported in this paper.

Data Availability

We aim for open research, reusability, and generalizability. The tool implementation, full dataset, and detailed experiment data are publicly available at `https://github.com/Septemberember/FSF-Slicer`.

Acknowledgment

This research work is supported by the supporting funds for talents of Nanjing University of Aeronautics and Astronautics.

CRediT authorship contribution statement

Yang Liu: Methodology, Software, Investigation, Validation, Formal analysis, Visualization, Writing – original draft, Writing – review & editing. **Zedong Jia:** Methodology, Software, Validation, Formal analysis, Writing – review & editing. **Xi Hu:** Software, Investigation, Validation, Writing – review & editing. **Dhawaj Kumar Tomar:** Methodology, Software, Writing – review & editing. **Ai Liu:** Methodology, Formal analysis, Supervision, Project administration, Writing – original draft, Writing – review & editing. **Shaoying Liu:** Methodology, Writing – review & editing.

References

Alqadi, B.S., Maletic, J.I., 2020. Integration of program slicing with cognitive complexity for defect prediction, in: Proceedings of ICSME 2020, Adelaide, Australia, IEEE. pp. 839–843. doi:10.1109/ICSME46990.2020.00106.

Baier, C., Katoen, J., 2008. Principles of model checking. MIT Press.

Canfora, G., Cimitile, A., Lucia, A.D., 1998. Conditioned program slicing. Inf. Softw. Technol. 40, 595–607. URL: https://doi.org/10.1016/S0950-5849(98)00086-X, doi:10.1016/S0950-5849(98)00086-X.

Chai, W., Yan, R., Zhang, W., Zhang, J., 2024. Slicing assisted program verification: An empirical study, in: Chin, W., Xu, Z. (Eds.), Theoretical Aspects of Software Engineering - 18th International Symposium, TASE 2024, Guiyang, China, July 29 - August 1, 2024, Proceedings, Springer. pp. 38–57. URL: https://doi.org/10.1007/978-3-031-64626-3_3, doi:10.1007/978-3-031-64626-3_3.

Chalupa, M., Strejcek, J., 2019. Evaluation of program slicing in software verification, in: Ahrendt, W., Tarifa, S.L.T. (Eds.), Integrated Formal Methods - 15th International Conference, IFM 2019, Bergen, Norway, December 2-6, 2019, Proceedings, Springer. pp. 101–119. URL: https://doi.org/10.1007/978-3-030-34968-4_6, doi:10.1007/978-3-030-34968-4_6.

Feng, Y., Li, S., 2023. Abstract interpretation, hoare logic, and incorrectness logic for quantum programs. Inf. Comput. 294, 105077. doi:10.1016/J.IC.2023.105077.

Galindo, C., Pérez, S., Silva, J., 2023a. Exception-sensitive program slicing. J. Log. Algebraic Methods Program. 130, 100832. doi:10.1016/j.jlamp.2022.100832.

Galindo, C., Pérez, S., Silva, J., 2023b. Program slicing of java programs. J. Log. Algebraic Methods Program. 130, 100826. doi:10.1016/j.jlamp.2022.100826.

Hoare, C.A.R., Wirth, N., 1973. An axiomatic definition of the programming language PASCAL. Acta Informatica 2, 335–355.

Jiang, S., Li, W., Li, H., Zhang, Y., Zhang, H., Liu, Y., 2012. Fault localization for null pointer exception based on stack trace and program slicing, in: Proceedings of QSIC 2012, Xi'an, Shaanxi, China, IEEE. pp. 9–12. doi:10.1109/QSIC.2012.36.

Jiang, S., Zhang, H., Wang, Q., Zhang, Y., 2010. A debugging approach for java runtime exceptions based on program slicing and stack traces, in: Proceedings of QSIC 2010, Zhangjiajie, China, IEEE Computer Society. pp. 393–398. doi:10.1109/QSIC.2010.23.

Lee, W.K., Chung, I.S., Yoon, G.S., Kwon, Y.R., 2001. Specification-based program slicing and its applications. J. Syst. Archit. 47, 427–443. URL: https://doi.org/10.1016/S1383-7621(01)00003-0, doi:10.1016/S1383-7621(01)00003-0.

Liu, A., Liu, S., 2023. Enhancing the capability of testing-based formal verification by handling operations in software packages. IEEE Trans. Software Eng. 49, 304–324. doi:10.1109/TSE.2022.3150333.

Liu, A., Liu, Y., Liu, S., Yang, Z., 2025. Testing-based formal verification with program slicing on functional soundness and completeness, in: Rümmer, P., Wu, Z. (Eds.), Theoretical Aspects of Software Engineering - 19th International Symposium, TASE 2025, Limassol, Cyprus, July 14-16, 2025, Proceedings, Springer. pp. 11–29. URL: https://doi.org/10.1007/978-3-031-98208-8_2, doi:10.1007/978-3-031-98208-8_2.

Liu, H., Liu, S., Liu, A., Fang, D., Xu, G., 2022. Verifying and improving neural networks using testing-based formal verification, in: Proceedings of SOFL+MSVL 2022, Madrid, Spain, Springer. pp. 126–141. doi:10.1007/978-3-031-29476-1_11.

Liu, H., Liu, S., Xu, G., Liu, A., Fang, D., 2024. NNTBFV: simplifying and verifying neural networks using testing-based formal verification. Int. J. Softw. Eng. Knowl. Eng. 34, 273–300. doi:10.1142/S0218194023500523.

Liu, S., 2012. Utilizing Hoare logic to strengthen testing for error detection in programs, in: Turing-100 - The Alan Turing Centenary, Manchester, UK, EasyChair. pp. 229–238. URL: https://easychair.org/publications/paper/476.

Liu, S., 2015. Automatic selection of system functional scenarios for formal specification animation, in: Proceedings of APSEC 2015, New Delhi, India, IEEE Computer Society. pp. 72–79. doi:10.1109/APSEC.2015.15.

Liu, S., 2016. A tool supported testing method for reducing cost and improving quality, in: Proceedings of QRS 2016, Vienna, Austria, IEEE. pp. 448–455. doi:10.1109/QRS.2016.56.

Liu, S., Hayashi, T., Takahashi, K., Kimura, K., Nakayama, T., Nakajima, S., 2010. Automatic transformation from formal specifications to functional scenario forms for automatic test case generation, in: Proceedings of the 9th SoMeT_10, Yokohama City, Japan, IOS Press. pp. 383–397. doi:10.3233/978-1-60750-629-4-383.

Liu, S., Nakajima, S., 2013. Combining specification-based testing, correctness proof, and inspection for program verification in practice, in: Proceedings of SOFL+MSVL 2013, Queenstown, New Zealand, Springer. pp. 3–16. doi:10.1007/978-3-319-04915-1_1.

Liu, S., Nakajima, S., 2022. Automatic test case and test oracle generation based on functional scenarios in formal specifications for conformance testing. IEEE Trans. Software Eng. 48, 691–712. doi:10.1109/TSE.2020.2999884.

O'Hearn, P.W., 2020. Incorrectness logic. Proc. ACM Program. Lang. 4, 10:1–10:32. doi:10.1145/3371078.

Peled, D.A., 2001. Software Reliability Methods. Texts in Computer Science, Springer. doi:10.1007/978-1-4757-3540-6.

Poskitt, C.M., 2021. Incorrectness logic for graph programs, in: Proceedings of ICGT 2021, Virtual Event, Springer. pp. 81–101. doi:10.1007/978-3-030-78946-6_5.

Rao, L., Liu, S., Liu, A., 2022. Testing program segments to detect runtime exceptions in java, in: Proceedings of SOFL+MSVL 2022, Madrid, Spain, Springer. pp. 93–105. doi:10.1007/978-3-031-29476-1_8.

Roggenbach, M., Cerone, A., Schlingloff, B., Schneider, G., Shaikh, S.A., 2022. Formal Methods for Software Engineering - Languages, Methods, Application Domains. Texts in Theoretical Computer Science. An EATCS Series, Springer. doi:10.1007/978-3-030-38800-3.

Schumann, J., 2001. Automated theorem proving in software engineering. Springer. URL: http://www.springer.com/computer/swe/book/978-3-540-67989-9.

Soremekun, E.O., Kirschner, L., Böhme, M., Zeller, A., 2021. Locating faults with program slicing: an empirical analysis. Empir. Softw. Eng. 26, 51. doi:10.1007/s10664-020-09931-7.

Wang, R., Liu, S., 2018. TBFV-SE: testing-based formal verification with symbolic execution, in: Proceedings of QRS 2018, Lisbon, Portugal, IEEE. pp. 59–66. doi:10.1109/QRS.2018.00019.

Wang, R., Liu, S., 2019. Branch sequence coverage criterion for testing-based formal verification with symbolic execution, in: Proceedings of QRS Companion 2019, Sofia, Bulgaria, IEEE. pp. 205–212. doi:10.1109/QRS-C.2019.00049.

Wang, R., Liu, S., Sato, Y., 2020. A fault localization approach derived from testing-based formal verification, in: Proceedings of ICECCS 2020, Singapore, IEEE. pp. 165–170. doi:10.1109/ICECCS51672.2020.00026.

Weiser, M.D., 1984. Program slicing. IEEE Trans. Software Eng. 10, 352–357. doi:10.1109/TSE.1984.5010248.

Yan, P., Jiang, H., Yu, N., 2022. On incorrectness logic for quantum programs. Proc. ACM Program. Lang. 6, 1–28. doi:10.1145/3527316.

Yin, Y., Liu, S., Chen, Y., 2018. Verification of sysml activity diagrams using hoare logic and SOFL, in: Duan, Z., Liu, S., Tian, C., Nagoya, F. (Eds.), Proceedings of SOFL+MSVL 2018, Gold Coast, QLD, Australia, Springer. pp. 71–88. URL: https://doi.org/10.1007/978-3-030-13651-2_5, doi:10.1007/978-3-030-13651-2_5.